

# SCOTS (0.2) – USER MANUAL

MATTHIAS RUNGGER  
MARCH 29, 2017

## CONTENTS

|                                          |           |
|------------------------------------------|-----------|
| 1. About SCOTS v0.2                      | 2         |
| 2. Quickstart                            | 2         |
| 2.1. Invariance and Reachability         | 2         |
| 2.2. Customized Synthesis                | 2         |
| 2.3. A Priori Enclosure and Growth Bound | 2         |
| 3. Installation Notes                    | 3         |
| 3.1. Additional software                 | 3         |
| <b>Part 1. THEORY BASICS</b>             | <b>5</b>  |
| 4. The Symbolic Approach                 | 5         |
| 4.1. Control Problems                    | 5         |
| 4.2. Supported Specifications            | 5         |
| 4.3. Auxiliary Control Problems          | 6         |
| 4.4. Closed Loop                         | 6         |
| 5. Construction of Symbolic Models       | 7         |
| 5.1. Growth Bound and A Priori Enclosure | 7         |
| 5.2. The Transition Function             | 8         |
| 6. Controller Synthesis                  | 8         |
| 6.1. Invariance                          | 9         |
| 6.2. Reachability                        | 9         |
| 6.3. Reach-Avoid                         | 10        |
| <b>Part 2. USAGE</b>                     | <b>11</b> |
| 7. Computation of Symbolic Models        | 11        |
| 8. Controller Synthesis                  | 13        |
| 9. Controller Implementation             | 15        |
| 10. Binary Decision Diagrams             | 16        |
| <b>Part 3. CASE STUDIES</b>              | <b>17</b> |
| 11. An Aircraft Landing Maneuver         | 17        |
| 12. A Vehicle in a Maze                  | 17        |
| 13. A DC-DC Boost Converter              | 18        |

## 1. ABOUT SCOTS v0.2

SCOTS is an open source software tool (available at <http://www.hcs.ei.tum.de>) published under the 3-Clause BSD License. It provides a basic implementation of the construction of symbolic models, also known as discrete abstractions, of possibly perturbed, nonlinear control systems according to [1] together with the implementation of two algorithms for the synthesis of symbolic controllers. It is mainly implemented in C++, but also provides a small MATLAB interface to access atomic propositions and the synthesized controllers from the MATLAB workspace.

SCOTS natively supports invariance and reachability specifications. It can also be used in combination with the synthesis tool `slugs` [2] to account for general reactivity one (GR(1)) specifications. Moreover, expert users have the possibility to write customized synthesis algorithms.

SCOTS is mainly intended to be used (and possibly extended) by researchers and lecturers in the area of formal methods for cyber-physical systems. The implementation does not use validated numerics or similar methods for rigorous implementations and such is prone to ODE solver inaccuracies and rounding errors.

This manual gives an overview of SCOTS and contains installation notes, usage details and a brief theoretical background. For implementation details please see the doxygen documentation in `./doc`.

Although, there are no compelling reasons, why SCOTS should not work under Windows, we developed and tested the code only under Linux and macOS environments. As a result, the installation notes apply to Linux/macOS systems only.

Bug reports and feature requests should be mailed to [matthias.rungger@tum.de](mailto:matthias.rungger@tum.de).

## 2. QUICKSTART

The best way to try SCOTS is to clone the source code from <https://gitlab.lrz.de/matthias/SCOTSV0.2> and run one of the examples. Each of the example directories contains a `readme` file that provides some background information on the example itself and explains the compilation process.

Optionally, some examples contain an `m-file` for the simulation of the closed loop in MATLAB. See <http://www.mathworks.com> for installation instructions.

**2.1. Invariance and Reachability.** The three most easy-to-compile examples are found in

---

```
./examples/dcdc /* invariance problem for a DCDC boost converter */
./examples/vehicle /* reach-avoid problem for a vehicle */
./examples/aircraft /* landing maneuver of an aircraft (requires 32GB memory) */
```

---

The examples can be run without any additional software and require only a C++ compiler with C++ 11 support. See also the case studies in PART 3

**2.2. Customized Synthesis.** In the examples in

---

```
./examples/dcdc_bdd /* reach-and-stay spec for a DCDC boost converter */
./examples/vehicle_bdd /* currently reachability only (more complex spec is planned) */
```

---

we use customized controller synthesis algorithms to enforce more complex specifications. Customized synthesis algorithms are supported only using binary decision diagrams (BDDs) as underlying data structure. In order to run the examples, an installation of the CUDD library is required, see Section 3.1.

**2.3. A Priori Enclosure and Growth Bound.** The directory

---

```
./examples/aircraft/helper
```

---

contains various programs that use the interval-arithmetic based ODE solver `vnode-lp` to obtain: **a)** an a priori enclosure and **b)** a growth bound.

### 3. INSTALLATION NOTES

SCOTS is implemented in “header-only” style and only a working C++ developer environment with C++ 11 support is needed. In the basic variant it is possible to

- (1) compute abstractions
- (2) synthesize controllers with respect to invariance and reachability specifications
- (3) simulate the closed loop in C++

For various reasons, one might consider to use additional software in combination with SCOTS.

#### 3.1. Additional software.

- (1) MATLAB: for closed loop simulation and visualization options.

See <http://www.mathworks.com> for installation instructions.

To access the controllers produced by SCOTS from the MATLAB workspace the mex file `mexStaticController.mex` needs to be compiled:

- (a) setup the mex compiler with the MATLAB command

```
>> mex -setup C++
```

- (b) In a terminal, navigate to `./mfiles/mexfiles`:

- edit the makefile and adjust the MATLABPATH
- run `make sparse`

To access the BDD files produced by SCOTS from the MATLAB workspace the mex file `mexSymbolicSet.mex` needs to be compiled:

- (a) install the CUDD library (see item 2)
- (b) setup the mex compiler with the MATLAB command

```
>> mex -setup C++
```

- (c) In a terminal, navigate to `./mfiles/mexfiles`:

- edit the makefile and adjust the MATLABPATH and CUDDPATH
- run `make bdd`

- (2) CUDD: **a)** to combine SCOTS with slugs, **b)** to save atomic propositions and transition relations as BDD to file and **c)** to write customized synthesis algorithms.

SCOTS uses CUDD with the following configuration options:

- the C++ object-oriented wrapper
- the dddmp library and
- the shared library

The package (available at <http://vlsi.colorado.edu/~fabio/>) follows the usual installation routine of `configure`, `make` and `make install`. We use `cudd-3.0.0`, with the configuration

```
./configure --enable-shared --enable-obj --enable-dddmp --prefix=CUDDPATH
```

where CUDDPATH is the desired installation directory, e.g., `./external/cudd`.

On some linux machines we experienced that the header files `util.h` and `config.h` were missing in CUDDPATH and we manually copied them to CUDDPATH/include.

**NOTE:** The `dddmp` module of CUDD, which is used to store BDDs to file, is erroneous and the following changes should be applied in CUDDROOT/dddmp/dddmpNodeBdd.c

---

```
232: + f->next = (DdNode *) (ptrueint) (((ptrueint) (f->next)) | 01ul);
232: - f->next = (DdNode *) (ptrueint) ((int) ((ptrueint) (f->next)) | 01);
256: + f->next = (DdNode *) (ptrueint) (((ptrueint) (f->next)) & (~01ul));
256: - f->next = (DdNode *) (ptrueint) ((int) ((ptrueint) (f->next)) & (~01));
409: + DddmpClearVisitedBdd (f);
409: - /*DddmpClearVisitedBdd (f);*/
410: + /*f->next = NULL;*/
410: - f->next = NULL;
```

---

- (3) `boost`: to use advance ode solvers.

For Linux `boost` is usually distributed via the package management system. On macOS MacPorts <https://www.macports.org/> or Homebrew <http://brew.sh/> provide an easy way to install `boost`.

- (4) `vnode-lp`: to compute a priori enclosures and growth bounds.

`vnode-lp` is an open source software tool to compute validated solutions of initial value problems based on interval arithmetic. It is available at .

## Part 1. THEORY BASICS

### 4. THE SYMBOLIC APPROACH

A detailed description of *the symbolic approach to controller synthesis* that is implemented in SCOTS is presented in [1]. The article [1] contains also a detailed explanation of the notation that we use in the following text.

**4.1. Control Problems.** SCOTS supports the computation of controllers for nonlinear control systems of the form

$$\dot{\xi}(t) \in f(\xi(t), u) + \llbracket -w, w \rrbracket \quad (1)$$

where  $f$  is given by  $f : \mathbb{R}^n \times U \rightarrow \mathbb{R}^n$  and  $U \subseteq \mathbb{R}^m$ . The vector  $w = [w_1, \dots, w_n] \in \mathbb{R}_+^n$  is a perturbation bound and  $\llbracket -w, w \rrbracket$  denotes the hyper-interval  $[-w_1, w_1] \times \dots \times [-w_n, w_n]$ . Given a time horizon  $\tau > 0$ , we define a *solution of (1) on  $[0, \tau]$  under (constant) input  $u \in U$*  as an absolutely continuous function  $\xi : [0, \tau] \rightarrow \mathbb{R}^n$  that satisfies (1) for almost every (a.e.)  $t \in [0, \tau]$ .

The desired behavior of the closed loop is defined with respect to the  $\tau$ -sampled behavior of the continuous-time systems (1). To this end, the sampled behavior of (1) is casted as *simple system* (with initial states)

$$S_1 := (X_1, X_{1,0}, U_1, F_1) \quad (2)$$

with the *state alphabet*  $X_1 := \mathbb{R}^n$ , the *set of initial states*, the *input alphabet*  $U_1 := U$  and the *transition function*  $F_1 : X_1 \times U_1 \rightrightarrows X_1$  defined by

$$F_1(x, u) := \{x' \mid \exists \xi \text{ is a solution of (1) on } [0, \tau] \text{ under } u : \xi(0) = x \wedge \xi(\tau) = x'\}. \quad (3)$$

A *specification*  $\Sigma_1$  for a simple system (2) is simply a set

$$\Sigma_1 \subseteq (U_1 \times X_1)^\infty := \bigcup_{T \in \mathbb{Z}_{\geq 0} \cup \{\infty\}} (U_1 \times X_1)^{[0;T[} \quad (4)$$

of possibly finite and infinite input-state sequences. A simple system  $S_1$  together with a specification  $\Sigma_1$  constitute an *control problem*  $(S_1, \Sigma_1)$ .

The *solution* of a control problem  $(S_1, \Sigma_1)$  is a system  $C = (X_c, X_{c,0}, U_c, V_c, Y_c, F_c, H_c)$  which is *feedback composable* with  $S_1$ , see [1, Def. III.3], and satisfies

$$\mathcal{B}(C \times S_1) \subseteq \Sigma_1.$$

In this context  $C$  and  $C \times S_1$  are usually referred to as *controller*, respectively, *closed loop*. The symbol  $\mathcal{B}(C \times S_1)$  denotes the *behavior* of the closed loop  $C \times S_1$ , see [1, Def. V.1]. We say that a control problem  $(S_1, \Sigma_1)$  is *solvable* iff there exists a system  $C$  that solves  $(S_1, \Sigma_1)$ .

A block diagram of the feedback composition of a controller synthesized with SCOTS and the system  $S_1$  is illustrated in Fig. 1.

**4.2. Supported Specifications.** SCOTS natively supports

- invariance (often referred to as safety) specifications;
- reachability specifications;
- reach-avoid specifications.

An *invariance* specification for (2) associated with  $Z_1 \subseteq X_1$  is defined by

$$\Sigma_1 := \{(u, x) \in (U_1 \times X_1)^{[0;\infty[} \mid \forall t \in [0;\infty[ : x(t) \in Z_1\}.$$

A *reachability* specification for (2) associated with  $Z_1 \subseteq X_1$  is defined by

$$\Sigma_1 := \{(u, x) \in (U_1 \times X_1)^\infty \mid \exists t \in [0;\infty[ : x(t) \in Z_1\}.$$

A *reach-avoid* specification for (2) associated with  $A_1 \subseteq X_1$  and  $Z_1 \subseteq X_1$  is defined by

$$\Sigma_1 := \{(u, x) \in (U_1 \times X_1)^\infty \mid \exists t \in [0;\infty[ : x(t) \in Z_1 \wedge \forall t' \in [0;t[ : x(t') \notin A_1\}.$$

In the context of Linear Temporal Logic, the sets  $A_1$  and  $Z_1$  are often identified with *atomic propositions*. In this sense, SCOTS allows to define arbitrary sets as atomic propositions.

**4.3. Auxiliary Control Problems.** Given a simple system (2) representing the  $\tau$ -sampled behavior of (1) and a specification  $\Sigma_1$  for (2), the control problem  $(S_1, \Sigma_1)$  is not solved directly, but an auxiliary, finite control problem  $(S_2, \Sigma_2)$  is used in the synthesis process. Here,

$$S_2 = (X_2, X_{2,0}, U_2, F_2) \quad (5)$$

is referred to as *symbolic model* or (discrete) *abstraction* of  $S_1$  and  $\Sigma_2$  is an *abstract specification*.

The state alphabet of  $X_2$  is a cover of  $X_1$  and the input alphabet  $U_2$  is a subset of  $U_1$ . The set  $X_2$  contains a subset  $\tilde{X}_2$ , representing the “real” quantizer symbols, while the remaining symbols  $X_2 \setminus \tilde{X}_2$  are interpreted as “overflow” symbols. The set of real quantizer symbols  $\tilde{X}_2$  are given by congruent hyper-rectangles aligned on a uniform grid

$$\eta\mathbb{Z}^n = \{c \in \mathbb{R}^n \mid \exists_{k \in \mathbb{Z}^n} \forall_{i \in [1;n]} c_i = k_i \eta_i\} \quad (6)$$

with *grid parameter*  $\eta \in (\mathbb{R}_+ \setminus \{0\})^n$ . The elements of  $\eta\mathbb{Z}^n$  are called *grid points*. The real quantizer symbols are further parameterized by two vectors  $a, b \in \mathbb{R}^n$  representing the lower-left and upper-right corners of the hyper-interval  $\llbracket a, b \rrbracket$  confining the set  $\tilde{X}_2$ :

$$\tilde{X}_2 := \{x_2 \mid \exists_{c \in (\eta\mathbb{Z}^n \cap \llbracket a, b \rrbracket)} x_2 = c + \llbracket -\eta/2, \eta/2 \rrbracket\}. \quad (7)$$

The elements of the real quantizer symbols are also referred to as *cells*. Each cell  $x_2 = c + \llbracket -\eta/2, \eta/2 \rrbracket$  is associated with a *center*  $c \in \mathbb{R}^n$  (which is also a *grid point*  $c \in \eta\mathbb{Z}^n$ ) and a *radius*  $r \in \mathbb{R}_{\geq 0}^n$ . The initial state alphabet  $X_{2,0}$  is defined by the cells that are needed to cover  $X_{1,0}$ , i.e.,

$$X_{2,0} := \{x_2 \in X_2 \mid x_2 \cap X_{1,0} \neq \emptyset\}. \quad (8)$$

SCOTS computes symbolic models that are related via feedback refinement relations with the plant. A *feedback refinement relation* from  $S_1$  to  $S_2$  is a strict relation  $Q \subseteq X_1 \times X_2$  that satisfies for all  $(x_1, x_2) \in Q$  and  $u \in U_2$  the conditions

- (1)  $x_1 \in X_{1,0}$  implies  $x_2 \in X_{2,0}$
- (2)  $F_2(x_2, u) \neq \emptyset$  implies  $F_1(x_1, u) \neq \emptyset$  and  $Q(F_1(x_1, u)) \subseteq F_2(x_2, u)$ .

In SCOTS, the feedback refinement relation  $Q$  is given by the set-membership relation

$$Q := \{(x_1, x_2) \mid x_1 \in x_2\}. \quad (9)$$

Given an invariance (reachability) specification  $\Sigma_1$  for (2) associated with  $Z_1$ , then an *abstract specification* associated with  $S_1$ ,  $S_2$  and  $Q$  is given by the invariance (reachability) specification for  $S_2 = (X_2, X_{2,0}, U_2, F_2)$  associated with

$$Z_2 = \{x_2 \in X_2 \mid x_2 \subseteq Z_1\}. \quad (10)$$

An abstract reach-avoid specification from  $A_1, Z_1$  for  $S_2$  follows by

$$A_2 = \{x_2 \in X_2 \mid x_2 \cap A_1 \neq \emptyset\} \quad (11)$$

and  $Z_2$  as defined in (10). The algorithms to solve the control problems  $(S_2, \Sigma_2)$  implemented in SCOTS are outlined in Section 6.

**4.4. Closed Loop.** The main statement facilitating the use of an auxiliary control problem reads as follows [1, Thm. VI.3]:

*Consider two control problems  $(S_i, \Sigma_i)$ ,  $i \in \{1, 2\}$ . Suppose that  $Q$  is a feedback refinement relation from  $S_1$  to  $S_2$  and  $\Sigma_2$  is an abstract specification of  $\Sigma_1$ . If  $C$  solves the control problem  $(S_2, \Sigma_2)$ , then  $C \circ Q$  solves the control problem  $(S_1, \Sigma_1)$ .*

The controller  $C \circ Q$  for  $S_1$  is given by the serial composition of the quantizer  $Q : X_1 \rightrightarrows X_2$  with the controller  $C$ . The closed loop resulting from a simple system  $\Sigma_1$  which represents the  $\tau$ -sampled behavior of (1) and a controller  $C \circ Q$  is illustrated in Fig. 1. At each  $k \in \mathbb{Z}_{\geq 0}$  sampling time  $\tau > 0$ , the plant state  $x_1 = \xi(k\tau)$  is measured and fed to the quantizer  $Q$ , which is used to determine a cell  $x_2 \in X_2$  that contains  $x_1 \in x_2$ . Then  $x_2$  is fed to the controller  $C$  to pick the input  $u \in U_2 \subseteq U_1$  which is applied to (1).

Additionally to the perturbations on the right-hand-side of (1), it is possible to account for measurement errors modeled by a set-valued map  $P : \mathbb{R}^n \rightrightarrows \mathbb{R}^n$  given by

$$P(x) := x + \llbracket -z, z \rrbracket \quad \text{with} \quad z \in \mathbb{R}_+^n. \quad (12)$$

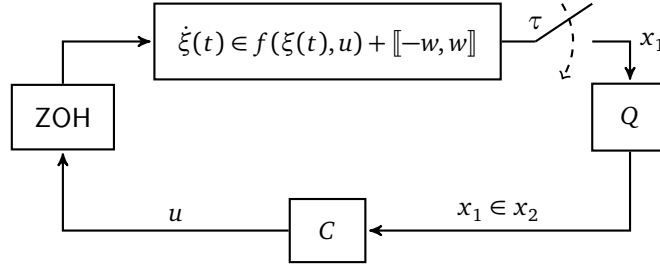
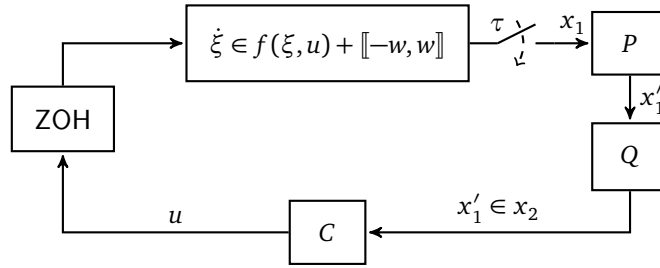


FIGURE 1. Sample-and-hold implementation of a controller synthesized with SCOTS.

Please see [1, Sec. VI.B] and [3] for some background theory. The closed loop with measurement errors is illustrated in Fig. 2.

FIGURE 2. Closed loop with measurement errors modeled by the set-valued map  $x'_1 \in P(x_1)$ .

## 5. CONSTRUCTION OF SYMBOLIC MODELS

**5.1. Growth Bound and A Priori Enclosure.** The construction of a symbolic model  $S_2$  of  $S_1$  is based on the over-approximation of attainable sets. In SCOTS, the over-approximation of the attainable sets requires a so-called growth bound [1]. A *growth bound* of (1) is a function  $\beta: \mathbb{R}_+^n \times U' \rightarrow \mathbb{R}_+^n$ , which is defined with respect to a sampling time  $\tau > 0$ , a set  $K \subseteq \mathbb{R}^n$  and a set  $U' \subseteq U$ . Basically, it provides an upper bound on the deviation of solutions  $\xi$  of (1) from *nominal solutions*<sup>1</sup>  $\varphi$  of (1), i.e., for every solution  $\xi$  of (1) on  $[0, \tau]$  with input  $u \in U'$  and  $\xi(0), p \in K$  we have

$$|\xi(\tau) - \varphi(\tau, p, u)| \leq \beta(|\xi(0) - p|, u). \quad (13)$$

Here,  $|x|$  for  $x \in \mathbb{R}^n$ , denotes the component-wise absolute value. Essentially, a growth bound can be obtained by bounding the Jacobian of  $f$ . Let  $L: U' \rightarrow \mathbb{R}^{n \times n}$  satisfy

$$L_{i,j}(u) \geq \begin{cases} D_j f_i(x, u) & \text{if } i = j, \\ |D_j f_i(x, u)| & \text{otherwise} \end{cases} \quad (14)$$

for all  $x \in K' \subseteq \mathbb{R}^n$  and  $u \in U' \subseteq U$ . Then

$$\beta(r, u) = e^{L(u)\tau} r + \int_0^\tau e^{L(u)s} w \, ds, \quad (15)$$

is a growth bound on  $[0, \tau]$ ,  $K, U'$  associated with (1). The set  $K'$  on which (14) needs to hold, is a so-called *a priori enclosure*, i.e.,  $K'$  is assumed to be convex and contain any solution  $\xi$  on  $[0, \tau]$  of (1) with  $u \in U'$  and  $\xi(0) \in K$ , see [1, Thm. VIII.5].

In order to use SCOTS, the user needs to provide a growth bound, which for nonlinear control systems can be provided in terms of the parameterized matrix  $L(u)$  whose entries satisfy (14). A priori enclosures as well as growth bounds can be computed automatically using interval arithmetic based ODE solvers. See Section 7 for more details on how to automatically obtain a priori enclosures and growth bounds.

<sup>1</sup>A nominal solution  $\varphi(\cdot, p, u)$  of (1) is defined as solution of the initial value problem  $\dot{x} = f(x, u)$ ,  $x(0) = p$ .

**5.2. The Transition Function.** Recall that the state alphabet  $X_2$  of the symbolic model (5) is composed of the real quantizer symbols  $\bar{X}_2$ , which are cells aligned on a uniform grid, and the overflow symbols  $X_2 \setminus \bar{X}_2$ . For  $x_2 \in X_2 \setminus \bar{X}_2$  the transition function is defined for all  $u \in U_2$  by

$$F_2(x_2, u) := \emptyset. \quad (16)$$

In order to determine the successors  $x'_2 \in F_2(x_2, u)$  for  $x_2 = c + \llbracket -\eta/2, \eta/2 \rrbracket \in \bar{X}_2$  and  $u \in U_2$ , we first compute the hyper-interval

$$R := \varphi(\tau, c, u) + \llbracket -\beta(\eta/2, u), \beta(\eta/2, u) \rrbracket \quad (17)$$

which is an over-approximation of the attainable set of (1) with respect to the set  $c + \llbracket -\eta/2, \eta/2 \rrbracket$  and input  $u$ . If  $P$  is not covered by the real quantizer symbols, i.e.,  $R \not\subseteq \cup_{x_2 \in \bar{X}_2} x_2$ , then we define

$$F_2(x_2, u) := \emptyset. \quad (18)$$

Otherwise, we define the successor cells function by

$$x'_2 \in F_2(x_2, u) : \iff x'_2 \cap R \neq \emptyset. \quad (19)$$

Using similar arguments as in [1, Thm. VIII.4], it is straightforward to show that  $Q$  is a feedback refinement relation from  $S_1$  to  $S_2$ . Note that  $F_2$  satisfies

$$F_2(x_2, u) \subseteq \bar{X}_2. \quad (20)$$

If we need to be robust against measurement errors  $P(x) = x + \llbracket -z, z \rrbracket$ , we slightly modify the computation of  $R$  to

$$R := \varphi(\tau, c, u) + \llbracket -\beta(\eta/2 + z, u), \beta(\eta/2 + z, u) \rrbracket \quad (21)$$

and define the transition function to (instead of (19))

$$x'_2 \in F_2(x_2, u) : \iff (x'_2 + \llbracket -z, z \rrbracket) \cap R \neq \emptyset. \quad (22)$$

As a result we obtain that  $Q \circ P$  is a feedback refinement relation from  $S_1$  to  $S_2$ , see [4, Thm. III.5], which enables the correct controller refinement under measurement errors [1, Sec. VI.B]. The use of the perturbation parameter  $z \in \mathbb{R}_{>0}^n$  in SCOTS is explained in detail in Section 7.

In the implementation of the computation of  $F_2$  in `Abstraction.hh`, we use a numerical ODE solver to compute an approximation of  $\varphi(\tau, c, u)$  as well as  $\beta(\eta/2, u)$ .

## 6. CONTROLLER SYNTHESIS

In this section, we discuss the algorithms that are implemented in SCOTS to solve synthesis problems for the finite symbolic model (5). Usually, synthesis algorithms are developed in the context of two-player games on graphs. The player associated with the controller tries to enforce the specification while the player associated with disturbances tries to violate the specification, e.g. [5].

Given a control problem  $(S_2, \Sigma_2)$  with  $S_2$  given in (5), the construction of a controller to enforce a specification  $\Sigma_2$  proceeds in two steps. First, a subset  $Y_\infty$  of the state alphabet  $X_2$  is computed, which characterizes the largest set of initial states so that the control problem is solvable, i.e.,  $(S_2, \Sigma_2)$  is solvable if and only if  $X_{2,0} \subseteq Y_\infty$ . The set  $Y_\infty$  is referred to as *winning domain* (or set of *winning states*) associated with  $(S_2, \Sigma_2)$ . In a second step, the controller  $C$  is derived from the set  $Y_\infty$  and some other information available in synthesis algorithm.

We use the following notation. The set of *admissible inputs* at  $x_2 \in X_2$  is denoted by

$$U_{S_2}(x_2) := \{u \in U_2 \mid F_2(x_2, u) \neq \emptyset\}. \quad (23)$$

We use the Weierstrass symbol  $\wp$  to denote the power set and define  $\text{pre} : \wp(X_2) \rightarrow \wp(X_2)$  by

$$\text{pre}(Y) := \{x_2 \in X_2 \mid \exists u \in U_{S_2}(x_2) : F_2(x_2, u) \subseteq Y\}. \quad (24)$$

**Algorithm 1** Controller synthesis for invariance specs associated  $Z_2$ **Input:**  $Z_2, S_2 = (X_2, U_2, F_2)$ **Require:**  $F_2(X_2, U_2) \subseteq \bar{X}_2$  and  $Z_2 \subseteq \bar{X}_2$ 

```

1:  $Q := \emptyset$  ▷ FIFO queue of bad states
2:  $E := \emptyset$  ▷ bookkeeping of the bad states
3:  $D := \emptyset$  ▷ valid state-input pairs
4: for all  $x_2 \in \bar{X}_2$  do
5:   if  $x_2 \notin Z_2$  or  $U_2(x_2) = \emptyset$  then ▷ mark all states outside  $Z_2$  or blocking states as bad
6:      $Q := Q \cup \{x_2\}$ 
7:   else
8:      $D := D \cup (\{x_2\} \times U_2(x_2))$ 
9:  $E := Q$ 
10: while  $Q \neq \emptyset$  do
11:    $x'_2 \leftarrow Q$  ▷ remove oldest element
12:    $Q := Q \setminus \{x'_2\}$ 
13:   for all  $(x_2, u) \in F_2^{-1}(x'_2)$  do
14:      $D := D \setminus \{(x_2, u)\}$  ▷ remove state-input pairs that lead to bad states
15:     if  $x_2 \notin D^{-1}(U_2)$  and  $x_2 \notin E$  then ▷ no valid input left and not already marked bad
16:        $Q := Q \cup \{x_2\}$  ▷ add to queue of bad states
17:        $E := E \cup \{x_2\}$ 

```

**Output:**  $D$ 

**6.1. Invariance.** Let  $\Sigma_2$  be an invariance specification associated with  $Z_2$ . We use Alg. 1, which is implemented in the function `solve_invariance_game` in the file `GameSolver.hh`, to synthesize a controller that solves the control problem  $(S_2, \Sigma_2)$ . Alg. 1 runs in  $O(m)$  time, where  $m$  is the number of transitions, i.e., the number of triples  $(x_2, u, x'_2)$  with  $x'_2 \in F_2(x_2, u)$ , since each state is added to the queue of bad states  $Q$  at most once.

Let  $D \subseteq X_2 \times U_2$  be the set of state-input pairs computed in Alg. 1. One can show that  $D^{-1}(U_2)$  is the *maximal fixed point* of the map  $G : \wp(X_2) \rightarrow \wp(X_2)$  defined by

$$G(Y) := Z_2 \cap \text{pre}(Y). \quad (25)$$

The maximal fixed point of (25) corresponds to the winning domain of  $(S_2, \Sigma_2)$  and it follows that the synthesis problem  $(S_2, \Sigma_2)$  is solvable if and only if  $X_{2,0} \subseteq D^{-1}(U_2)$ .

Suppose that  $X_{2,0} \subseteq D^{-1}(U_2)$  holds, then we obtain a controller  $C = (\{q\}, \{q\}, X_2, X_2, U_2, F_c, H_c)$  that solves  $(S_2, \Sigma_2)$  by

$$\begin{aligned}
H_c(q, x_2) &= \begin{cases} D(x_2) \times \{x_2\} & \text{if } x_2 \in D^{-1}(U_2) \\ U_2 \times \{x_2\} & \text{otherwise} \end{cases} \\
F_c(q, x_2) &= \begin{cases} \{q\} & \text{if } x_2 \in D^{-1}(U_2) \\ \emptyset & \text{otherwise.} \end{cases}
\end{aligned} \quad (26)$$

The refined controller that solves  $(S_1, \Sigma_1)$  is given by  $C \circ Q$ . Note that the definition of the initial state alphabet  $X_{2,0}$  in (8) and the condition  $X_{2,0} \subseteq D^{-1}(U_2)$  imply that the initial state alphabet of  $S_1$  satisfies

$$X_{1,0} \subseteq \cup_{x_2 \in D^{-1}(U_2)} X_2. \quad (27)$$

The controller  $C \circ Q$  is implemented in SCOTS in the `StaticController` class. The details are explained in Section 9.

**6.2. Reachability.** Let  $\Sigma_2$  be a reachability specification associated with  $Z_2 \subseteq \bar{X}_2$ . We use Alg. 2, which is implemented in the function `solve_reachability_game` in the file `GameSolver.hh`, to synthesize a controller that solves the control problem  $(S_2, \Sigma_2)$ . The algorithm is a variant of Dijkstra's shortest path algorithm for hyper-graphs taken from [6]. It runs in  $O(m)$  time, where  $m$  is the number of transitions, i.e., the number of elements in  $X_2 \times U_2 \times X_2$  that satisfy  $x'_2 \in F_2(x_2, u)$ .

**Algorithm 2** Controller synthesis for reachability specs associated with  $Z_2$ **Input:**  $Z_2, S_2 = (X_2, U_2, F_2), u_0 \in U_2$ **Require:**  $Z_2 \subseteq \bar{X}_2$ 

```

1:  $Q := Z_2$  ▷ FIFO queue
2:  $V := \infty$  ▷ value function
3:  $M := 0$  ▷ intermediate values
4:  $E := \emptyset$  ▷ bookkeeping of processed states
5: for all  $x_2 \in \bar{X}_2$  do
6:    $D(x_2) := \emptyset$  ▷ keep track of optimal input
7:   if  $x_2 \in Z_2$  then
8:      $V(x_2) := 0$  ▷ cost at target are zero
9:      $D(x_2) := \{u_0\}$ 
10: while  $Q \neq \emptyset$  do
11:    $x'_2 := Q$  ▷ remove oldest element
12:    $Q := Q \setminus \{x'_2\}$ 
13:    $E := E \cup \{x'_2\}$ 
14:   for all  $(x_2, u) \in F_2^{-1}(x'_2)$  do
15:      $M(x_2, u) := \max\{M(x_2, u), V(x'_2)\}$ 
16:     if  $F_2(x_2, u) \subseteq E$  and  $V(x_2) > 1 + M(x_2, u)$  then ▷ if  $u$  leads to better cost update input
17:        $V(x_2) := 1 + M(x_2, u)$ 
18:        $Q := Q \cup \{x_2\}$ 
19:        $D(x_2) := \{u\}$ 

```

**Output:**  $D, V$ 

Let  $D$  be the output of Alg. 2. The set  $D^{-1}(U_2)$  is the *minimal fixed point* of the map  $G : \wp(X_2) \rightarrow \wp(X_2)$  defined by

$$G(Y) := Z_2 \cup \text{pre}(Y). \quad (28)$$

The minimal fixed point of (28) corresponds to the winning domain of  $(S_2, \Sigma_2)$  and it follows that the synthesis problem  $(S_2, \Sigma_2)$  is solvable if and only if  $X_{2,0} \subseteq D^{-1}(U_2)$ .

Given that  $X_{2,0} \subseteq D^{-1}(U_2)$  holds, a controller  $C = (\{q\}, \{q\}, X_2, X_2, U_2, F_c, H_c)$  that solves  $(S_2, \Sigma_2)$  is identical to the controller that solves the invariance problem, i.e.,  $F_c$  and  $H_c$  are given by (26).

Again, the refined controller  $C \circ Q$  that is feedback composed with  $S_1$  is implemented in SCOTS in the `StaticController` class. The details are explained in Section 9.

**6.3. Reach-Avoid.** Let  $\Sigma_2$  be a reach-avoid specification associated with  $A_2, Z_2 \subseteq \bar{X}_2$ . SCOTS provides two alternatives to synthesize controllers to enforce reach-avoid specifications. In the first method, the avoid set  $A_2$  is accounted for in the transition function, i.e.,

$$\forall_{x_2 \in A_2} : F_2(x_2, u) := \emptyset. \quad (29)$$

In this way we can reduce a reach-avoid problem to a reachability problem. Given that the transition function satisfies (29), any controller that solves the control problem  $(S_2, \Sigma'_2)$ , where  $\Sigma'_2$  is the reachability specification associated with  $Z_2$ , also solves the control problem  $(S_2, \Sigma_2)$ . In order to enforce the equation (29) in the computation of the transition function  $F_2$  the user can optionally supply the avoid set to the function `Abstraction::compute_gb`, see Section 7.

For the second method, the computation of the transition function is left unchanged, but Alg. 2 is modified to account for the avoid set. To this end the condition in line 16 in Alg. 2 is modified to

$$F_2(x_2, u) \subseteq E \quad \text{and} \quad V(x_2) > 1 + M(x_2, u) \quad \text{and} \quad x_2 \notin A_2. \quad (30)$$

Again the user can optionally supply the avoid set to the function `solve_reachability_game`, see Section 8.

## Part 2. USAGE

### 7. COMPUTATION OF SYMBOLIC MODELS

Let  $S_1$  be a simple system that represents the  $\tau$ -sampled behavior of the continuous-time system (1) as defined in (2). In order to compute the transition function  $F_2$  of a symbolic model (5) of  $S_1$  in SCOTS, the following ingredients are needed.

- (1) The solution  $\varphi(\tau, x, u)$  of the IVP  $\dot{\xi} = f(\xi, u)$ ,  $\xi(0) = x$  at time  $\tau$ , which is provided in terms of a lambda expression of the form

---

```
[(state_type& x, const input_type& u) -> void {}]
```

---

The state solution of the IVP at time  $\tau$ , i.e.,  $\varphi(\tau, x, u)$  is expected to be stored in the variable  $x$ . The `state_type` and `input_type` are aliases for `std::array<double, dim>` in the appropriate dimension.

- (2) A growth bound  $\beta(r, u)$ , which is provided by a lambda expression of the form

---

```
[(state_type& r, const state_type& c, const input_type& u) -> void {}]
```

---

Again, the value of the growth bound  $r' = \beta(r, u)$  is assumed to be stored in the variable  $r$ . In the implementation, the growth bound is allowed to depend the center of the cell  $c$ . See [4] for details.

- (3) The set of real quantizer symbols  $\bar{X}_2$  as defined in (7), which is defined as instantiation of the class with constructor

---

```
template<class grid_point_t>
scots::UniformGrid(const int dim,
                   const grid_point_t& lb,
                   const grid_point_t& ub,
                   const grid_point_t& eta)
```

---

where the variables `dim`, `lb`, `ub` and `eta` represent the state space dimension, the lower and upper bound of the hyper-interval confining the set  $\bar{X}_2$ , and the grid parameter, respectively.

Internally, each cell  $x_2 = c + \llbracket -\eta/2, \eta/2 \rrbracket \in \bar{X}_2$  is represented as an unsigned integer of type `abs_type`. In the default setting, `abs_type` is an alias set in `UniformGrid.hh` to

---

```
using abs_type=std::uint32_t;
```

---

which limits the size of  $\bar{X}_2$  to  $2^{32} - 1$ . The template member functions

---

```
template<class grid_point_t>
scots::UniformGrid::itox(abs_type id, state_type& x)
template<class grid_point_t>
scots::UniformGrid::abs_type xtoi(const grid_point_t& x) const
```

---

are useful in mapping an abstract state  $c + \llbracket -\eta/2, \eta/2 \rrbracket \in \bar{X}_2$ , represented by its unsigned integer ID `id`, to its center  $c \in \mathbb{R}^n$  aligned on the uniform grid (6). Similarly, `xtoi` can be used to map a state  $x \in \mathbb{R}^n$  to the id of the cell  $x_2 \in \bar{X}_2$  containing it, i.e.,  $x \in x_2$ .

- (4) The input alphabet  $U_2$ , which is again represented by an instance of a `UniformGrid`.
- (5) Optionally, it is possible to provide a measurement error bound to account for measurement errors as defined in (12) using the member function

---

```
scots::Abstraction::set_measurement_error_bound(const state_type& z)
```

---

We demonstrate the usage of SCOTS to compute a symbolic model of the sampled dynamics of an aircraft used in [1, Sec. IX.B]. The system consists of three states  $x_1, x_2, x_3$ , which respectively correspond to the velocity, the flight path angle and the altitude of the aircraft. The input alphabet is given by  $U = [0, 160 \cdot 10^3] \times [0^\circ, 10^\circ]$  and represents the thrust of the engines (in Newton) and the angle of attack. Additionally, we use the disturbance vectors  $w = (0.108, 0.002, 0)^\top$  in (1) and  $z = (0.0125, 0.0025^\circ, 0.05)^\top$  in (12) to model possible input disturbances, respectively, measurement errors.

For the computation of the transition function, we begin with the aliases for the `state_type` and `input_type` by

---

```
const int state_dim=3; /* state space dim */
const int input_dim=2; /* input space dim */
using state_type = std::array<double,state_dim>;
using input_type = std::array<double,input_dim>;
```

---

Afterwards, we define the lambda for the solution of the IVP for a sampling time of  $\tau = 0.25$  by

---

```
auto aircraft_post = [](state_type &x, const input_type &u) {
    /* the ode describing the aircraft */
    auto rhs = [](state_type& xx, const state_type &x, const input_type &u) {
        double mg = 60000.0*9.81;
        double mi = 1.0/60000;
        double c=(1.25+4.2*u[1]);
        xx[0] = mi*(u[0]*std::cos(u[1])-(2.7+3.08*c*c)*x[0]*x[0]-mg*std::sin(x[1]));
        xx[1] = (mi/x[0])*(u[0]*std::sin(u[1])+68.6*c*x[0]*x[0]-mg*std::cos(x[1]));
        xx[2] = x[0]*std::sin(x[1]);
    };
    /* use 10 intermediate steps */
    scots::runge_kutta_fixed4(rhs,x,u,state_dim,tau,10);
};
```

---

where we use the fixed step-size ODE solver `scots::runge_kutta_fixed4` that ships with SCOTS. An implementation that uses a more advanced ODE solver like the adaptive step-size solvers with error control implemented in `boost` can be found in `./examples/aircraft_boost`.

We continue with the definition of the growth bound, for which we determine the matrix  $L(u)$  in (14). First, we compute an a priori enclosure  $K'$  of (1) with respect to  $\tau = 0.25$ ,  $U' = U$  and the hyper-interval  $P(K)$  where  $K = [58, 83] \times [-3^\circ, 0] \times [0, 56]$  is the hyper-interval that confines the set of real quantizer symbols. To this end, we employ the interval arithmetic based ODE solver implemented in `vnnode-lp` to obtain

$$K' = [57.55, 83.23] \times [-4.29^\circ, 1.22^\circ] \times [-1.38, 56.28].$$

The implementation of the computation of the a priori enclosure can be found in the example directory `./examples/aircraft/helper/a_priori_enclosure`. To obtain the matrix  $L(u)$ , we maximize the partial derivatives of  $f(\cdot, u)$  over the set  $K'$ . The solution of the linear IVP  $\dot{r}(t) = L(u)r(t) + w$ ,  $r(0) = r_0$ , at time  $\tau$  provides the growth bound (15) that we access through the lambda

---

```
auto radius_post = [](state_type &r, const state_type &, const input_type &u) {
    /* the ode for the growth bound */
    auto rhs = [](state_type& rr, const state_type &r, const input_type &u) {
        /* lipschitz matrix */
        double L[3][2];
        L[0][0]=-0.001919*(2.7+3.08*(1.25+4.2*u[1])*(1.25+4.2*u[1]));
        L[0][1]=9.81;
        L[1][0]=0.002933+0.004802*u[1];
        L[1][1]=0.00361225;
        L[2][0]=0.07483;
        L[2][1]=83.22;
        /* to account for input disturbances */
        state_type w={{0.108,0.002,0}};
        rr[0] = L[0][0]*r[0]+L[0][1]*r[1]+w[0];
        rr[1] = L[1][0]*r[0]+L[1][1]*r[1]+w[1];
        rr[2] = L[2][0]*r[0]+L[2][1]*r[1]+w[2];
    };
    /* use 10 intermediate steps */
    scots::runge_kutta_fixed4(rhs,r,u,state_dim,tau,10);
};
```

---

The real quantizer symbols  $\bar{X}_2$  and the input alphabet  $U_2$  are introduced as instances of the class `UniformGrid` by

---

```
state_type s_lb={{58,-3*M_PI/180,0}};
state_type s_ub={{83,0,56}};
state_type s_eta={{25.0/362,3*M_PI/180/66,56.0/334}};
scots::UniformGrid ss(state_dim,s_lb,s_ub,s_eta);

input_type i_lb={{0,0}};
input_type i_ub={{32000,8*M_PI/180}};
input_type i_eta={{32000,8.0/9.0*M_PI/180}};
scots::UniformGrid is(input_dim,i_lb,i_ub,i_eta);
```

---

The grid parameter `s_eta` for the real quantizer symbols  $\bar{X}_2$  has been determined according to the optimization procedure in [4].

For the computation of the symbolic transition function  $F_2$  we instantiate the class `Abstraction` and set the measurement error bound to  $z$

---

```
scots::Abstraction<state_type,input_type> abs(ss,is);
state_type z={{0.0125,0.0025/180*M_PI,0.05}};
abs.set_measurement_error_bound(z);
```

---

The computation of  $F_2$  itself is invoked by

---

```
abs.compute_gb(tf,aircraft_post,radius_post);
```

---

Note that, due to the numerical errors in the computation of the solution of the IVP  $\varphi(\tau, x, u)$  it is possible that the computed transition function  $F_2$  does not satisfies the requirements in Sectin 5.2. Similarly, the computation of the growth bound  $\beta(r, u)$  might be erroneous. Therefore, it is crucial to pick adequate ODE solver parameters to obtain high confidence in the correctness of the computation.

We list the runtimes of the computation of  $F_2$  for two different solvers for various parameters in Table 7. The implementations can be found in the directories `./examples/aircraft_boost` and `./examples/aircraft`.

| adaptive step size runge_kutta_dopri5 |                     |                     | fixed step size runge_kutta_4 |         |
|---------------------------------------|---------------------|---------------------|-------------------------------|---------|
| abs_tol/rel_tol                       |                     |                     | # intermediate steps          |         |
| $10^{-10}/10^{-10}$                   | $10^{-12}/10^{-12}$ | $10^{-16}/10^{-16}$ | 5                             | 10      |
| 252 sec                               | 370 sec             | 1036 sec            | 233 sec                       | 335 sec |

TABLE 1. Runtimes to compute a symbolic transition function of the aircraft dynamics with varying ODE solvers and solver parameters. All computations resulted in an identical transition function with  $5.86 \cdot 10^9$  transitions.

## 8. CONTROLLER SYNTHESIS

Let  $\Sigma_2$  be an invariance or reachability specification associated with  $Z_2 \subseteq \bar{X}_2$  for (5). The algorithms Alg. 1 and Alg. 2 to compute the winning domain  $D^{-1}(U_2)$  associated with the control problem  $(S_2, \Sigma_2)$  are implemented in the following functions.

- (1) Alg. 1 is implemented `solve_invariance_game` with signature

---

```
template<class F>
WinningDomain solve_reachability_game(const TransitionFunction& tf, F& ap)
```

---

where the set  $Z_2$  is defined as lambda expression `ap` of the form

---

```
[] (const scots::abs_type& abs_state) -> bool {}
```

---

`ap(abs_state)` returns true if and only if the abstract state  $x_2 \in \bar{X}_2$ , represented by the ID `abs_state`, is an element of the safe set  $Z_2$ .

- (2) Alg. 2 is implemented in `solve_reachability_game` with signature

---

```
template<class F>
WinningDomain solve_reachability_game(const TransitionFunction& tf, F& ap)
```

---

where the target set  $Z_2$  is defined as lambda expression `ap` of the form

---

```
[] (const scots::abs_type& abs_state) -> bool {}
```

---

`ap(abs_state)` returns `true` if and only if the abstract state  $x_2 \in \bar{X}_2$ , represented by the ID `abs_state`, is an element of the safe set  $Z_2$ . Additionally, a lambda expression as above can be supplemented to `solve_reachability_game` to define the avoid set  $A_2$ .

- (3) Optionally, the IDs of the cells contained in the set  $Z_2$  together with the uniform grid information can be stored to a file using the function
- 

```
template<class F>
bool write_to_file(const UniformGrid& g, F& ap, const std::string& filename)
```

---

The grid points associated with the stored IDs are accessible from the MATLAB workspace using the command

---

```
>> p = GridPoints('filename');
```

---

In order to use this MATLAB command, the mex-file `GridPoints.mex` needs to be compiled. Please see the installation notes for details.

We illustrate the usage of those functions again by the aircraft example studied in the previous section. Let (2) represent the  $\tau$ -sampled behavior of the aircraft dynamics and consider the reachability specification  $\Sigma_1$  for  $S_1$  associated with the set

$$Z_1 := ([63, 75] \times [-3^\circ, 0^\circ] \times [0, 2.5]) \cap \{x \in \mathbb{R}^3 \mid x_1 \sin x_2 \geq -0.91\}.$$

Consider the system (5), the quantizer (9) and the perturbation map (12). As outlined in Section 4.3, an abstract specification associated with  $S_1$ ,  $S_2$  and  $Q \circ P^2$  is a reachability specification for  $S_2$  associated with  $Z_2$ , where  $Z_2$  is required to be a subset of  $\{x_2 \in X_2 \mid P(x_2) \subseteq Z_1\}$ . We represent  $Z_2$  as lambda target as follows

---

```
/* define target set */
state_type t_lb = {{63, -3*M_PI/180, 0}};
state_type t_ub = {{75, 0, 2.5}};
state_type c_lb;
state_type c_ub;
state_type x;
auto target = [&](const scots::abs_type abs_state) {
    /* compute the center of cell associated with abs_state */
    ss.itox(abs_state, x);
    /* hyper-interval of the quantizer symbol with perturbation */
    for(int i=0; i<state_dim; i++) {
        c_lb[i] = x[i]-s_eta[i]/2.0-z[i];
        c_ub[i] = x[i]+s_eta[i]/2.0+z[i];
    }
    if( t_lb[0]<=c_lb[0] && c_ub[0]<=t_ub[0] &&
        t_lb[1]<=c_lb[1] && c_ub[1]<=t_ub[1] &&
        t_lb[2]<=c_lb[2] && c_ub[2]<=t_ub[2]) {
        if(-0.91<=(x[0]*std::sin(x[1])-
            s_eta[0]/2.0-z[0]-(c_ub[0])*(s_eta[1]/2.0-z[1]))) {
            return true;
        }
    }
    return false;
}
```

---

<sup>2</sup>In the computation of the abstraction specification, we replace  $Q$  with  $Q \circ P$  to account for the measurement errors, see [1, Sec. VI.B].

---

```
};
```

---

Once we have the lambda `target` representing the target set, we compute the winning domain of  $(S_2, \Sigma_2)$  by

---

```
WinningDomain win=solve_reachability_game(tf,target);
```

---

The number of states in the winning domain and the winning domain itself is obtainable by

---

```
abs_type domain = win.get_size();
std::vector<abs_type> domain = win.get_winning_domain();
```

---

It is possible to store the result as `StaticController` to a file. The syntax for storing the controller to the file `controller.scs` is given by

---

```
write_to_file(StaticController(ss,is,std::move(win)),"controller");
```

---

For various reasons it might be helpful to visualize the target set. To this end, we write the IDs of the grid points in the target set together with the uniform grid information to a file

---

```
write_to_file(ss,target,"target");
```

---

Subsequently, we access and plot the grid points in the MATLAB workspace by

---

```
>> p = GridPoints('target');
>> plot3(p(:,1),p(:,2),p(:,3),'.');
```

---

## 9. CONTROLLER IMPLEMENTATION

Let  $C$  be the static controller defined in (26) that solves  $(S_2, \Sigma_2)$ . For the implementation of the refined controller  $C \circ Q$  illustrated in Fig. 1 (or Fig. 2), SCOTS provides the class `StaticController`. The class can be instantiated with two `UniformGrids` and a `WinningDomain` according to the constructor

---

```
scots::StaticController(const UniformGrid& state_grid,
                        const UniformGrid& input_grid,
                        WinningDomain&& winning_domain)
```

---

Alternatively, it is possible to instantiate the `StaticController` class with the default constructor and subsequently read the content of the controller from file by

---

```
inline bool read_from_file(StaticController& sc, const std::string& filename)
```

---

To extract the control inputs associated with a state  $x \in X_1$  SCOTS provides the member function

---

```
template<class state_type, class input_type>
std::vector<input_type> scots::StaticController::get_control(const state_type &x)
```

---

Internally, the mapping  $Q$  in Fig. 1 is implemented by the member function `UniformGrid::xtoi`, which maps  $x$  to the ID `abs_state` of the nearest grid point  $c \in \mathbb{R}^n$ . The cell that is associated with that grid point  $c$  contains  $x$ , i.e.,  $x \in c + \llbracket -\eta/2, \eta/2 \rrbracket$ . If  $x$  is not an element of the winning domain  $\cup_{x_2 \in D^{-1}(U_2)} X_2$ , then  $F_c(q, Q(x)) = \emptyset$  and no progress is possible. This is indicated by the error message

---

```
scots::StaticController: state <x> is out of winning domain: no progress possible.
```

---

For the aircraft example, we implemented a closed loop simulation in the file `./aircraft/simulate.cc`. We read the controller from file and simulate the system until the target is reached

---

```
/* read controller from file */
scots::StaticController con;
if(!read_from_file(con,"controller")) {
    std::cout << "Could not read controller from controller.scs\n";
    return 0;
}
/* initial state */
```

---

```

state_type x={{81, -1*M_PI/180, 55}};
while(1) {
    std::vector<input_type> u = con.get_control<state_type,input_type>(x);
    std::cout << x[0] << " " << x[1] << " " << x[2] << "\n";
    aircraft_post(x,u[0]);
    if(target(x))
        break;
}
return 0;

```

---

In order to access the StaticController from the MATLAB workspace SCOTS provides the MATLAB class StaticController. The data can be loaded by

---

```
>> con = StaticController('controller')
```

---

The inputs associated with a state  $x \in \mathbb{R}^n$  and the center of cells in the winning domain  $D^{-1}(U_2)$  are accessed by

---

```
>> u = con.control(x);
>> Y = con.domain;
```

---

In order to be able to use those commands, you need to add the directory `./mfiles` to your MATLAB path. A demonstration of the usage of the StaticController from the MATLAB workspace can be found in the various m-files in the example directories.

## 10. BINARY DECISION DIAGRAMS

SCOTS provides a number of classes and functions to obtain a *binary decision diagram* (BDD) representation of symbolic models, controllers and atomic propositions.

Useful to

- (1) save memory;
- (2) use SCOTS in combination with other tools, such as slugs and SENSE;
- (3) compactly save atomic propositions and controllers to a file;
- (4) write customized synthesis algorithms;

For usage details have look in the examples

---

```

./examples/dcdc_bdd /* reach-and-stay */
./examples/vehicle_bdd /* reachability */
./examples/aircraft_bdd /* reachability */

```

---

and have a look in the class documentation in `./doc/html/` of

---

```

scots::SymbolicSet
scots::SymbolicModel

```

---

and the companion functions `read_from_file` and `write_to_file`.

### Part 3. CASE STUDIES

#### 11. AN AIRCRAFT LANDING MANEUVER

We consider an aircraft DC9-30. The dynamics (1) according to [7] are given by

$$\begin{aligned}\dot{\xi}_1 &= \frac{1}{m}(u_1 \cos u_2 - D(u_2, \xi_1) - mg \sin \xi_2) \\ \dot{\xi}_2 &= \frac{1}{mx_1}(u_1 \sin u_2 + L(u_2, \xi_1) - mg \cos \xi_2) \\ \dot{\xi}_3 &= \xi_1 \sin \xi_2\end{aligned}$$

where  $D(u_2, x_1) = (2.7 + 3.08 \cdot (1.25 + 4.2 \cdot u_2)^2) \cdot x_1^2$ ,  $L(u_2, x_1) = (68.6 \cdot (1.25 + 4.2 \cdot u_2)) \cdot x_1^2$  and  $mg = 60 \cdot 10^3 \cdot 9.81$  account for the drag, lift and gravity, respectively [7]. The state variables  $x_1$ ,  $x_2$  and  $x_3$  correspond to the velocity, the flight path angle and the altitude of the aircraft. The input alphabet is given by  $U = [0, 32 \cdot 10^3] \times [0^\circ, 8^\circ]$  and represents the thrust of the engines (in Newton) and the angle of attack. Following [1], we consider certain input disturbance, which we model in (1) by  $w = (0.108, 0.002, 0)^\top$  and measurement errors of the form (12) with  $z = \frac{1}{20}(0.25, 0.05, 1)^\top$ .

The objective is to control the aircraft from a certain height close to the ground. We cast this controller problem as a reachability problem for which the target set is given by

$$Z_1 := ([63, 75] \times [-3^\circ, 0^\circ] \times [0, 2.5]) \cap \{x \in \mathbb{R}^3 \mid x_1 \sin x_2 \geq -0.91\}.$$

For the computation of the symbolic model, we fix a sampling time of  $\tau = 0.25$  sec and restrict the problem domain to  $\llbracket a, b \rrbracket$  with

$$a = (58, -3^\circ, 0)^\top \text{ and } b = (83, 0^\circ, 56)^\top$$

and set  $\eta = (25.0/362, 3^\circ/66, 56.0/334)^\top$ . The real quantizer symbols  $\bar{X}_2$  are determined according to (7). The set of inputs is given by the grid points  $U_2 := \mu\mathbb{Z}^2 \cap U$  with  $\mu = (32 \cdot 10^3, (8/9)^\circ)^\top$ . The choice of discretization parameter leads to a successful solution of the reachability problem. The height of the controlled aircraft over time is illustrated in Fig. 3.

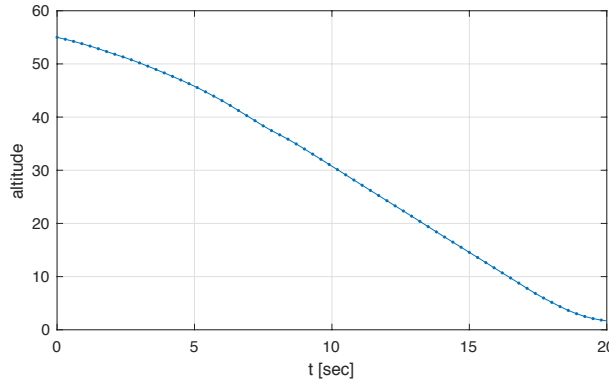


FIGURE 3. Trajectory of the controlled aircraft landing maneuver.

The configuration files for the computation in SCOTS can be found in `./examples/aircraft`.

#### 12. A VEHICLE IN A MAZE

In this case study we consider the vehicle illustrated in Fig. 4. We follow [8] and use the bicycle dynamics to model the vehicle and obtain

$$\begin{aligned}\dot{\xi}_1 &= u_1 \cos(\alpha + \xi_3) \cos(\alpha)^{-1} \\ \dot{\xi}_2 &= u_1 \sin(\alpha + \xi_3) \cos(\alpha)^{-1} \\ \dot{\xi}_3 &= u_1 \tan(u_2)\end{aligned}$$

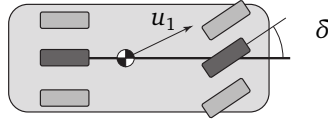


FIGURE 4. Vehicle illustration taken from [8].

where  $\alpha = \arctan(\tan(u_2)/2)$ . The states  $(x_1, x_2)$  and  $x_3$  correspond to the position, respectively, orientation of the vehicle in the 2-dimensional plane. The inputs are given by the velocity  $u_1 \in [-1, 1]$  and the rate  $u_2 \in [-1, 1]$  of the steering angle  $\delta$ .

The specification is illustrated in Fig. 5. The goal is to synthesize a controller that steers the vehicle through the maze, without hitting the blue obstacles until the target set (red box) is reached. The synthesis problem is casted as a reach-avoid problem.

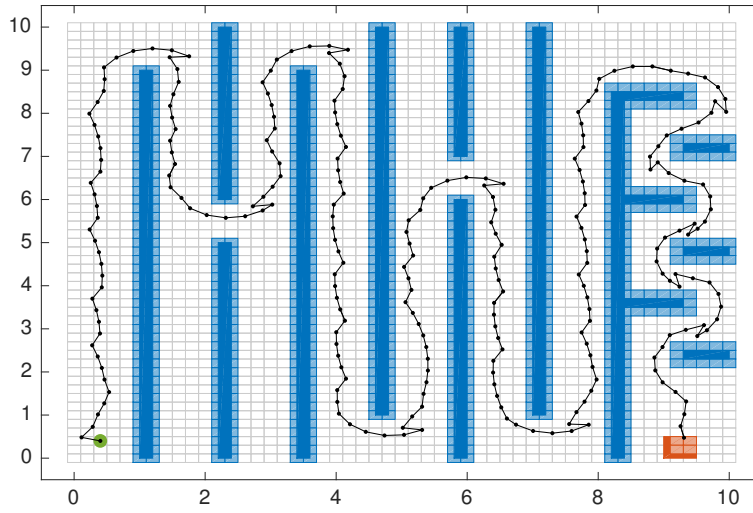


FIGURE 5. Trajectory of the controlled aircraft landing maneuver.

For the computation of the symbolic model, we fix the sampling time to  $\tau = 0.3$  sec and limit the problem domain to  $\llbracket a, b \rrbracket$  with

$$a = (0, 0, -3.5)^\top \text{ and } b = (10, 10, 3.5)^\top$$

and set  $\eta = (0.2, 0.2, 0.2)^\top$ . The real quantizer symbols  $\bar{X}_2$  are determined according to (7). The set of inputs is given by the grid points  $U_2 := \mu\mathbb{Z}^2 \cap U$  with  $\mu = (0.3, 0.3)^\top$ . A trajectory of the closed loop resulting from the controller synthesized with those choices of parameters is illustrated in Fig. 5.

The configuration files for the computation in SCOTS using the sparse matrix implementation and the BDD based implementation can be found in `./examples/vehicle` and `./examples/vehicle_bdd`, respectively.

### 13. A DC-DC BOOST CONVERTER

A DC-DC boost converter (illustrated in Fig. 6) is a step-up converter which is used to generate a higher voltage on the load side compared to the source voltage. The converter operates in two modes. When the switch is closed, electrical energy is stored in terms of a magnetic field in the inductor. When the switch is open, the inductor acts as a second source and its voltage adds up to the source voltage on the load side. DC-DC boost converters have been extensively studied in the literature, often from a hybrid control perspective, and also in the context of abstraction-based controller design [9–11].

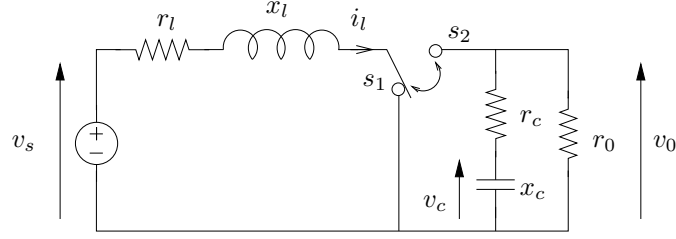


FIGURE 6. The DC-DC boost converter.

The state of the system is given by  $x(t) = [i_l(t), v_c(t)]^\top$  where  $i_l(t)$  and  $v_c(t)$  denote the current through the inductor and the voltage across the capacitor at time  $t$ , respectively. The dynamical behavior is described by the switched linear system

$$\dot{x}(t) = A_{g(t)}x(t) + b$$

with  $A_i \in \mathbb{R}^{2 \times 2}$ ,  $i \in [1; 2]$  and  $b$  given by

$$A_1 = \begin{bmatrix} -\frac{r_l}{x_l} & 0 \\ 0 & -\frac{1}{x_c} \frac{1}{r_0 + r_c} \end{bmatrix}, A_2 = \begin{bmatrix} -\frac{1}{x_l} (r_l + \frac{r_c r_0}{r_c + r_0}) & -\frac{1}{x_l} \frac{r_0}{r_0 + r_c} \\ \frac{1}{x_c} \frac{r_0}{r_0 + r_c} & -\frac{1}{x_c} \frac{1}{r_0 + r_c} \end{bmatrix}, b = \begin{bmatrix} \frac{v_s}{x_l} \\ 0 \end{bmatrix}.$$

First, we synthesize a controller to enforce the safety specification associated with

$$Z_1 := [1.15, 1.55] \times [5.45, 5.85].$$

In the computation of the abstraction, we fix the sampling time to  $\tau = 0.5$  sec and pick a grid point distance of  $\eta = (5 \cdot 10^{-4}, 5 \cdot 10^{-4})^\top$ . The winning domain together with a trajectory of the closed loop is illustrated in Fig. 7. The implementation details are provided in `./examples/dcdc` using the sparse matrix representation.

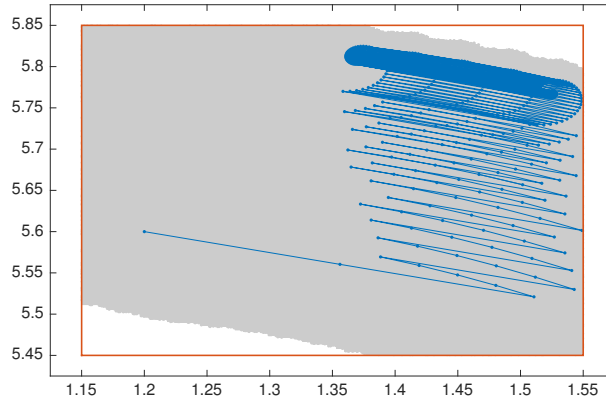


FIGURE 7. The domain of the invariance controller together with a closed loop trajectory.

We continue with a demonstration on how to use the BDD representation together with the CUDD Library to operate directly on the transition function and atomic propositions in order to create customized synthesis algorithms. In particular, let us consider the *reach-and-stay* specification  $\Sigma_1$  associated with  $Z_1 \subseteq X_1$  given by

$$\Sigma_1 := \{(u, x) \in (U_1 \times X_1)^\infty \mid \exists t \in [0; \infty[ \forall t' \in [t; \infty[ : x(t') \in Z_1\}.$$

The associated abstract specification  $\Sigma_2$  for the symbolic model  $S_2$  is given by the reach-and-stay specification associated with

$$Z_2 := \{x_2 \in X_2 \mid x_2 \subseteq Z_1\}.$$

We can solve the control problem  $(S_2, \Sigma_2)$  by the nested fixed point algorithm

$$\mu \bar{Y}. \nu Y. (\text{pre}(Y) \cap Z_2) \cup \text{pre}(\bar{Y}).$$

We implemented the fixed point computation in `./examples/bdd/dcdc_bdd` by using the enforceable predecessor computed in the class `scots::Enf_Pre` according to

---

```

/* set up enf_pre computation */
scots::EnfPre enf_pre(manager,TF,sym_model);
/* outer fp*/
BDD Z=manager.bddOne();
BDD ZZ=manager.bddZero();
/* inner fp*/
BDD Y=manager.bddZero();
BDD YY=manager.bddOne();
/* the controller */
BDD C=manager.bddZero();
/* helper */
BDD U=ss_input.get_cube(manager);
/* as long as not converged */
for(size_t i=1; ZZ != Z; i++) {
    Z=ZZ;
    BDD preZ=enf_pre(Z);
    /* init inner fp */
    YY = manager.bddOne();
    for(size_t j=1; YY != Y; j++) {
        Y=YY;
        YY = ( enf_pre(Y) & S ) | preZ;
    }
    ZZ=YY;

    /* only add (state/input) pairs whose state is not already in the * controller */
    BDD N = ZZ & (!(C.ExistAbstract(U)));
    C=C | N;
}

```

---

The resulting controller is stored in the BDD `C`. A closed loop trajectory of the system is illustrated in Fig. 8. The trajectory is obtained by running the MATLAB file `dcdc.m` in the example directory `./examples/bdd/dcdc_bdd`.

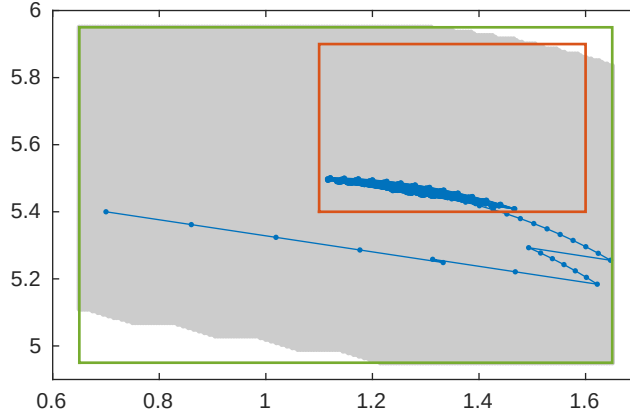


FIGURE 8. Closed loop trajectory of the DC-DC boost converter satisfying the reach-and-stay specification associated with the red rectangle.

## REFERENCES

- [1] G. Reißig, A. Weber, and M. Rungger. “Feedback Refinement Relations for the Synthesis of Symbolic Controllers”. In: *IEEE Transactions on Automatic Control* (2015). DOI: [10.1109/TAC.2016.2593947](https://doi.org/10.1109/TAC.2016.2593947).
- [2] R. Ehlers and V. Raman. “Slugs: extensible GR(1) synthesis”. In: *International Conference on Computer Aided Verification*. Springer, 2016, pp. 333–339.
- [3] M. Rungger and M. Zamani. “SCOTS: A Tool for the Synthesis of Symbolic Controllers”. In: *HSCC*. ACM, 2016.
- [4] A. Weber, M. Rungger, and G. Reissig. “Optimized State Space Grids for Abstractions”. In: *IEEE Transactions on Automatic Control* (2017). DOI: [10.1109/TAC.2016.2642794](https://doi.org/10.1109/TAC.2016.2642794).
- [5] R. Bloem et al. “Synthesis of reactive (1) designs”. In: *Journal of Computer and System Sciences* 78.3 (2012), pp. 911–938.
- [6] G. Gallo et al. “Directed hypergraphs and applications”. In: *Discrete Applied Mathematics* 42 (1993), pp. 177–201.
- [7] I. Mitchell, A. M. Bayen, and C. J. Tomlin. “Validating a Hamilton-Jacobi Approximation to Hybrid System Reachable Sets”. In: *Proc. 4th Intl. Conf. on Hybrid Systems: Computation and Control*. Springer, 2001, pp. 418–432.
- [8] K. J. Aström and R. M. Murray. *Feedback systems: an introduction for scientists and engineers*. Princeton University Press, 2010.
- [9] A. Girard, G. Pola, and P. Tabuada. “Approximately bisimilar symbolic models for incrementally stable switched systems”. In: *IEEE TAC* 55.1 (2010), pp. 116–126.
- [10] A. Girard. “Controller synthesis for safety and reachability via approximate bisimulation”. In: *Automatica* 48.5 (2012), pp. 947–953.
- [11] G. Reißig and M. Rungger. “Abstraction-based Solution of Optimal Stopping Problems under Uncertainty”. In: *Proc. of the 52nd IEEE Conf. on Decision and Control*. 2013, pp. 3190–3196. DOI: [10.1109/CDC.2013.6760370](https://doi.org/10.1109/CDC.2013.6760370).